
SECTION 1 PYTHON PRIMER

Structure

- 1.0 Introduction
- 1.1 Objectives
- 1.2 Python Language Overview
- 1.3 Data Types in Python
- 1.4 Classes and Objects in Python
- 1.5 Aliasing in Python
- 1.6 Expressions in Python
- 1.7 Control Flow
- 1.8 Looping / Repetition
- 1.9 Functions
- 1.10 Built-in functions
- 1.11 Taking Multiple inputs from Console
- 1.12 Exception Handling
- 1.13 Built-in Exceptions in Python
- 1.14 Iterators and Generators
- 1.15 Generators in Python
- 1.16 Additional Python Conveniences
- 1.17 Scopes and Namespaces
- 1.18 Summary
- 1.19 Further Readings

1.0 INTRODUCTION

Any data to be stored or retrieved from our computer requires usage of data structures. Creating and using these data structures can be done with plethora of high level programming languages like C, C++, Java or Python. In recent years, Python has become a language of choice since it has simple syntax, easier code readability, user friendly and can be used with wide variety of applications. It has extensive library, large active user community, cross-platform compatibility and beginner-friendly design philosophy. To understand its acceptability, for web and software development, Python is behind many websites and applications. For understand the hidden data patterns, Data analysts use Python's libraries to process and visualize huge datasets. In the fields of machine learning and artificial intelligence, Python is vastly used. Python's journey began with its first release, Python 0.9.0, in 1991 although it has undergone significant enhancements, with Python 2.0 introducing list comprehensions and garbage collection. A major transition occurred with the introduction of Python 3.0 in 2008 that made it more versatile.

Most of the high level languages are compiled but a few are interpreted. In a compiled language, the high level code is translated into machine code or executable code with the aid of a compiler before execution. On the other hand, interpreter reads each line of code, interprets it into machine instructions, and executes it straight away. There isn't any need of separate compilation. Python code is interpreted language through Python interpreter. The interpreter receives a command, evaluates that command, and reports the result of the command. The source code in Python is conventionally stored

in a file named with the .py suffix like hello.py. These programs are executed by the interpreter.

1.1 OBJECTIVES

The purpose of this section is to introduce and fly you across the world of Python to the extent possible.

After studying this section, you should be able to understand :

- Syntax of Python
- Different operators in Python

It is also strongly recommended that, in parallel, register to the free courses of Python which are given in references at the end of this section.

1.2 PYTHON LANGUAGE OVERVIEW

Python is a dynamic, interpreted or a byte code-compiled language. This means that prototyping can be very quick while testing the code. As you code in Python, you shall look at it like procedural at times or object-oriented. One doesnot have to bother about the type declarations of variables, parameters, functions, or methods in code you write. Python interpreter tracks the types of all values at runtime and flags code that does not make sense when it runs. Python code works on variety of platforms like Windows, Mac, Linux, Raspberry Pi, etc. It has a simple syntax quite similar to the English language. It requires lesser lines as compared to other high level programming languages because of its rich library.

1.2.1 How to install Python on Windows

To install Python on Windows systems, one can visit the python.org download page and download the current Python 3.X.X version. Now, Run the Python installer and accept all the defaults that are displayed. This will install Python in the root directory. Now the complete Python is installed and one can open the command prompt from:

Accessories > Command Prompt, or

type cmd into the run dialog

```
# Your First Python program
```

```
# This program prints Welcome to Python
```

```
Print('Welcome to Python')
```

Note that we use the built-in `print()` function to print the string Welcome to Python on the screen.

1.3 DATA TYPES IN PYTHON

Data types define the type of a variable that represents the type of data that can be stored in a variable and the kind of operations that can be done on it. The table below lists the builtin data types in Python.

Data Type	Description
Numeric	int, float, complex
Sequence	list, tuple range
Mapping	dict
Set	set, frozenset
Text	str
Binary	bytes, bytearray, memoryview

Numeric Types:

int: Used to represents whole numbers (e.g.: 2, -15, 0).

float: Used to represents decimal numbers (e.g.: 2.14, -0.8, 8.0).

complex: Used to represents complex numbers in the form $a + bk$, where a and b are floats and k is the imaginary section (e.g.: $6 + 8k$, $-1k$).

Sequence Types:

str: Used to represents text as a sequence of characters (e.g.: "good", 'world', "987").

list: Used to represents an ordered, mutable (modifiable) sequence of items, which can be of different types (e.g., $[1, \text{"gold"}, 2.77]$).

tuple: Used to represents an ordered, immutable (un-modifiable) sequence of items (e.g.: $[10, \text{"amul"}, 3.14]$).

range: Used to represents a sequence of numbers (e.g.: `range(3)` generates numbers from 0 to 2).

Mapping Type:

dict: Used to represents a collection of key-value pairs, where keys are unique and immutable (e.g.: `{"name": "Karan", "age": 27}`).

Set Types:

set: Used to represents an unordered collection of unique items (e.g., $\{2,4,8\}$).

frozenset: Used to represents an immutable version of a set.

Boolean Type:

bool: Used to represents truth values that can have two states True or False.

Binary Types:

bytes: Used to represents immutable sequence of bytes.

bytearray: Used to represents mutable sequence of bytes.

memoryview: Provides us with a view of an object's memory.

1.4 CLASSES AND OBJECTS IN PYTHON

Python is an object-oriented language and classes form the basis for all its data types. Classes are a means of bringing together data and functionality together. Each class encapsulates data and behaviour (methods) into a single entity. This structure allows you to model real-world objects and create organized, reusable code. A class in Python serves as a template for creating objects, which are instances of the class or instantiated from the class. We use classes when we need to encapsulate related data and functions, making the code modular and easier to manage. After defining a class, one can create multiple objects that share the same attributes and methods, while all these objects have their own unique state.

Example:

class car:

 name = ""

 Colour = ""

An object is an instance of a class. Creating a new instance of a class is known as instantiation. Hence Car is a class and now we can create objects like car1, car2 etc from this class as under:

Class car:

 name = "BMW"

 Colour = "white"

1.4.1 Identifiers, Objects, and the Assignment Statement

Identifiers

In Python, an identifier is a name given to a programming element, such as a variable, function, class, or module. It allows programmers to refer to these elements within their code. On the other hand, a variable, is an entity whose value can change over time. It denotes a memory location where a value can be stored. Naming rules for identifiers require these to begin with a letter or underscore, followed by letters, underscores, or digits. Additionally, it's essential to avoid using reserved words as identifiers. The first letter of an identifier cannot be a digit. These are case sensitive and hence "variable" and "Variable. Are two distinct identifiers". Whitespaces are not allowed in the identifiers. The scope of identifiers can be either local or global, dictating their accessibility within the program.

Reserved Words

Python 3 has 33 while Python 2 has 30 reserved words. The reserved word print was removed from Python 2 keywords and added as a built-in Python function.

False	def	if	raise
None	del	import	return
True	elif	in	try
and	else	is	while
as	except	lambda	with
assert	finally	nonlocal	yield
break	for	not	
class	from	or	
continue	global	pass	

Table 1: List of Python reserved words in Python that can't be used as identifiers.

1.5 ALIASING IN PYTHON

Each identifier is associated with the memory address of the object to which it referring to. An identifier can be associated with one type of object initially and later it can be reassigned to another object that is of the same or different type.

```
x = [10, 20]
y = x           # y now refers to the same list object as x
print(x is y)   # Output: True (since x and y are aliases)
```

Let us say, we decide to add another number to the list then the scenario is:

```
y.append(30)    # Modifying y also affects x

print(x)        # Output: [10, 20, 30]
print(y)        # Output: [10, 20, 30]
```

In the example illustrated above, x and y are aliases because they point to the same list. Modifying any one of the the list will affect the other. These characteristics should be borne in mind when working with mutable objects like lists and dictionaries. Aliasing can lead to unexpected side effects if not handled carefully. To avoid any unintended effect, one can create a copy of the object instead of creating an alias.

```
x = [10, 20]

y = x.copy()    # y is now a copy of x

print(x is y)   # Output: False (x and y are different objects)

y.append(30)
```

```
print(x) # Output: [10, 20]
print(y) # Output: [10, 20, 30]
```

1.6 EXPRESSIONS IN PYTHON

Like in C or C++, a combination of operands and operators is called an expression. The expression produces some value or result after being interpreted by the Python interpreter. It combines operators, variables, literals, and function calls to produce a value. The programming language uses distinct expressions for various requirements. Decision-making requires Boolean values from logical expressions like and, or. As you know, relational expressions compare values and return Booleans in Python, while arithmetic expressions compute numbers.

Complex expressions may involve numerous operators and types. The sequence of operation is determined by the Operator precedence in these expressions. A defined operator precedence hierarchy in Python ensures proper expression evaluation. Programmers have additional control by changing the evaluation order with parentheses. The program efficiency depends on using expression types and operator precedence rules.

Table below illustrates various operators from the highest to the lowest precedence along with its description and the associativity of the operator.

Precedence	Operator	Description	Operator Associativity
1	()	Parentheses	Left to right
2	x[index], x[index:index]	Subscription, slicing	Left to right
3	await x	Await expression	N/A
4	**	Exponentiation	Right to left
5	+x, -x, ~x	Positive, negative, bitwise NOT	Right to left
6	*, @, /, //, %	Multiplication, matrix, division, floor division, remainder	Left to right
7	+, -	Addition and subtraction	Left to right
8	<<, >>	Shifts	Left to right
9	&	Bitwise AND	Left to right
10	^	Bitwise XOR	Left to right
11		Bitwise OR	Left to right
12	in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, membership tests, identity tests	Left to Right
13	not x	Boolean NOT	Right to left
14	and	Boolean AND	Left to right
15	or	Boolean OR	Left to right
16	if-else	Conditional expression	Right to left
17	lambda	Lambda expression	N/A
18	:=	Assignment Operator	Right to Left

Illustrating Operator precedence

```
value = 2 + 4 * 6
```

```
print (value)
```

➔ Output of print statement shall be 26. This is because multiplication operation has higher precedence and hence $4 * 6$ gets computed first as 24. Later 24 is added to 2 to produce 26 as output.

Illustrating Operator Associativity

```
value = 2 * 4 / 6
```

```
print (value)
```

It can be seen here that both $*$ and $/$ have same precedence but the associativity tells us that the expression shall be evaluated from left to right. Hence Multiplication is performed first and later division.

➔ Output of print statement shall be 0.48.

Constant Expressions

An expression in Python that contains only constant values is known as a constant expression. In a **constant expression**, the operator(s) is a constant. A constant is a value that cannot be changed after its initialization.

Example:

```
val = 2 + 4    # both 2 and 4 are constants although val is a variable
```

➔ Output is 6

1.6.1 Integral Expressions

An integral expression in Python is used for computations only after necessary type conversion (integer to float, a string to integer, etc.).

Example:

```
x = 2          #x is assigned with 2 which is an integer number
```

```
y = 4.0        #y is assigned with 4.0 which is a floating point number
```

In order to carry out addition of these two variables, either we convert int to float and add two float numbers or we convert float to int and add two int numbers.

```
val = x + int(y)    # y type converted to int before addition operation
```

➔ Output is 6

or

```
val = float(x) + y    # x type converted to float before addition operation
```

➔ Output is 6.0

1.6.2 Relational Expressions

These are used in expression evaluation to compare two operands and return a Boolean value that is either True or False based on the evaluation of the expression. These operators find use in decision flow and conditional logic in programming used in if statements and loops.

Common relational operators in Python are listed below:

Less than	<	(e.g., $x < y$)
Less than or equal to	<=	(e.g., $x <= y$)
Greater than	>	(e.g., $x > y$)
Greater than or equal to	>=	(e.g., $x >= y$)
Equal to	==	(e.g., $x == y$)
Not equal to	!=	(e.g., $x != y$)

Illustrating relational expressions

```
x = 2
y = 4

print(x < y)      # Output: True
print(x <= 5)     # Output: True
print(x >= y)     # Output: False
print(y == 4)    # Output: True
print(x != y)    # Output: True
```

1.6.3 Logical Expressions

These performs the logical computation and the value of the overall expression results in either True or False i.e. boolean in nature. These operators are crucial for controlling program flow and making decisions based on multiple conditions in the expression. Python has three main logical operators namely and, or, and not.

```
x = 20
```

```
print(x > 2 and x < 25)
```

This returns True because 20 is greater than 2 AND 20 is less than 25.

or Operator

```
x = 100
```

```
print(x > 50 or x < 20)
```

This returns True because one of the conditions are true.

#100 is greater than 50, but 100 is not less than 20.

not Operator

```
x = 10
```

```
print(not(x > 2 and x < 25))
```

```
# This returns False because not operator reverses the result of and.
```

1.7 CONTROL FLOW

Program execution happens sequentially in Python wherein the Python interpreter reads a code written by you line by line from top to bottom with each statement interpreted from left to right and. The interpreter executes operations and functions in the order that it reads which is what the control flow signifies. Not every program shall follow this pattern since the program shall also encounter conditional statement and based on this outcome, the control gets transferred to the desired statement. Without control flow expressions, a program is simply a list of statements that are sequentially executed. Python uses if, elif (short for else if), and else keywords to create control flow structures in your program. Imagine all these keywords are traffic controllers telling the control flow where to go. Thus a program's control flow decides the order in which the program code executes. Python has three types of control structures which are listed as under:

Sequential: Default mode if no other branch logic implemented

Selection: Used for decisions in conditional statements and branching

Repetition: Iteration with the looping where a section of code gets executed the said number of times.

1.7.1 Sequential Statements

These are the statements where execution happens in a sequence. The problem with sequential statements is that if the logic has broken in any of the lines, the complete source code execution will fail.

```
# The following piece of code represents a Sequential statement
```

```
a=20
b=10
c=a-b
print("Subtraction of the two values is : ",c)
```

➔ Output

Subtraction of the two values is: 10

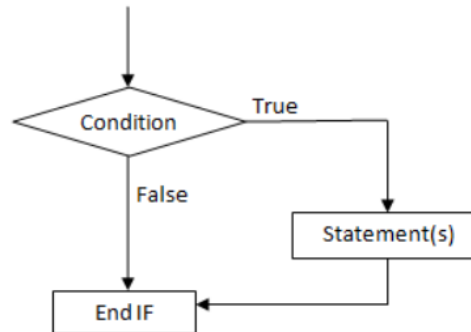
1.7.2 Decision Control Statements

When a program code statement or set of statements is executed, a condition evaluates to True or False is referred to as Decision Control Structure. Whether a section of code is executed or not is decided by checking a condition. Some decision control statements are as listed as under:

- if
- if-else
- nested if
- if-elif-else

if Statement

The if statement is a basic decision control statement used in decision making. It helps to run a particular code only when a certain condition is true. The same is illustrated by a figure below.



Program to increment a number if it is positive

```
n = 100          # Initialize the value of n
if (n>50):        # test the value of n
    n = n+1       # Increment the value of n if n is > 50
print(n)          # print the value of
```

→ Output

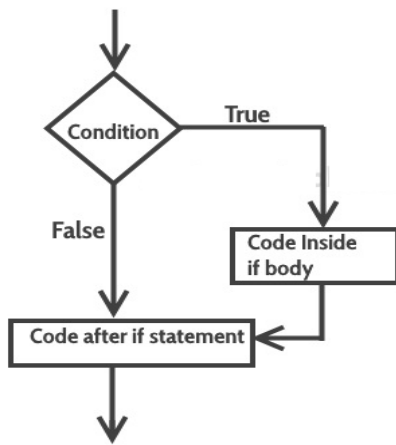
101

if-else

If-else statement is a control structure that ensures a program to execute different blocks of code based on the evaluation of a condition that evaluates to true or false. It checks a condition, and if it's true, the code block within the if statement is executed and if the condition is false, the code block after the else statement is executed. One should remember that Python uses indentation in both the blocks to define the scope of the code. Other programming languages often use curly brackets {} for this purpose. The same is illustrated below.

```
if(condition):
    #Executes this block if the condition is true
else:
    #Executes this block if the condition is false
```

The flow of decision control is illustrated in the figure below.



Program to increment a number if it is positive

```
marks = 71
if marks > 60:
    print("First Division")
else:
    print("Second Division ")
n is odd
```

➔ Output

First Division

Nested if or if .. elif .. else Statement

Nested **if** statements in Python allow you to have an if statement within another if statement. This is useful for scenarios where you need to evaluate multiple conditions during the execution of the program. These are an efficient way for coding complex / nested decision-making logic in your programs. Remember that one must ensure proper indentation and keep the code readable, understandable and maintainable. The same is illustrated here.

```
if (condition1):
    #Program Statements
elif (condition2):
    #Program Statements
. elif (condition3):
    #Program Statements
else:
    #Program Statements
```

Compare the two numbers using nested if (elif) statements

```
x = 100          # value 100 assigned to x
y = 200          # value 200 assigned to y
if x > y:
    print("x is greater than y")
elif:
    if x < y:
        print("y is greater than x")
    else:
```

```
print("x and y are equal")
```

➔ Output

y is greater than x

1.8 LOOPING / REPETITION

Understanding loops is essential for any python programmer as these provide a powerful tool to automate code logic and process data. Python being popular programming language, offers several loop structures to iterate over sequences and execute repetitive tasks efficiently. These are also referred to as repetition statement used to repeat a block of code statements. Python has two loops structures which are:

for loop

while loop

1.8.1 for loop

It is used to iterate over a sequence that can be a list, tuple, dictionary, or a set. It executes the block of code for each item in the sequence until the last item in sequence. This loop starts with the for keyword, followed by a variable that represents the current item in the sequence. The 'in' keyword links the variable to the sequence you want to iterate over. A colon ':' is used at the end of the loop header, and the indented block of code is executed once for each item in the sequence.

for item in sequence:

Code block to be executed for each item

Let us understand this with a sample program code illustrated below.

Program to find the sum of all elements in a given list

```
elements = (1,2,3,4,5)
total = 0
for item in elements:
    total = total + item
print ("Total of all elements in the given list =", total)
```

➔ Output

Total of all elements in the given list = 15

Let us get a brief insight into the following python code snippet, it illustrates how to use a for loop to iterate over a list and access its elements, while the second example shows how to iterate over a range of numbers.

Program to iterate on a given list and print elements of a given list

```
givenlist = [1, 2, 3]
for i in range(len(givenlist)):
    print(givenlist [i], end = " \n")
```

➔ Output

```
1
2
3
```

Program to iterate on a range of numbers

```
for elements in range(0,4):
    print(elements, end = " \n")
```

➔ Output

```
0
1
2
3
```

Note that the end parameter in the print() function specifies the character or string that should be printed at the end of the output. The default value is “\n” (newline character), ensuring each value to be printed on a new line.

1.8.2 while and do-while loop

The while loop in Python executes a body or block of code repeatedly until the condition is true. It is useful when the number of iterations is not known to the programmer in advance. The condition is evaluated before each iteration over the block of code. If it is true, the code block inside the loop is executed. This process continues until the condition becomes false. One should ensure to add certain termination condition like break.

while condition:

Body / Program Code to be executed repeatedly

Python code to illustrate while loop where the code iterates from 0 to 4 and prints each value using . Finally, it prints “End of list” to signify the end of the program after the loop is completed.

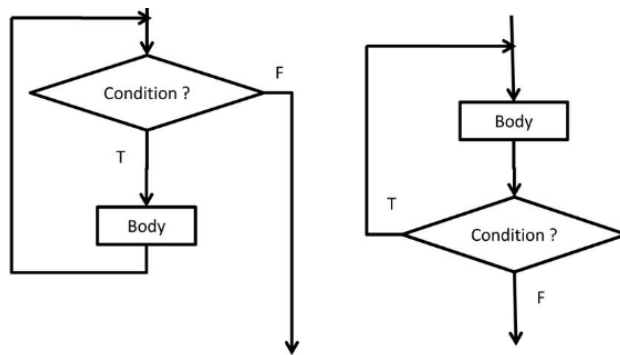
```
number = 5
i = 0
while i < number:
    print(i, end = " ")
    i = i + 1
print("End of List")
```

➔ Output

```
0 1 2 3 4 End of List
```

It is worth mentioning that Python does not have a built-in do-while loop like other programming languages. However, it is possible to exhibit the similar effect of a do-while loop using a while loop along with a break statement.

The only difference worth remembering is that in a do-while loop, the code block is executed at least once, and then the condition is checked. Thus, if the condition is true, the loop continues else it terminates. The same is illustrated in the figure below.



Program to illustrate do-while loop and its termination

```
number = 1
while True:
    print(number)
    number = number + 1
    if number > 10:
        break
```

→ Output

```
1
2
3
4
5
6
7
8
9
10
```

1.9 FUNCTIONS

A function is a block of reusable code that performs a specific task. This task is generally one which is used multiple times to develop an application in Python. Suppose we need to create a program to compute area of a given land or the volume of a box. The idea is to organize code, make it more readable, and avoid repetition of the same code statements. Thus, it helps to divide a complex problem into smaller ones and makes our program code easy to understand and reuse. Let's take a quick glimpse of how a function works.

Creating a Function

```
def welcomemessage():

    print("welcome to the world of Python")
```

The keyword “def” is used to create a function. welcomemessage () is the name of the function. The print statement forms the function body. Now in order to use or call this function, the following program gives you the insight.

program to define a Function and call it

```
def welcomemessage ():
```

```
    print("welcome to the world of Python")
```

```
welcomemessage()
```

➔ Output

```
welcome to the world of Python
```

Lets take a more relevant program that computes area. It takes two parameters. The length and the breath. Later, a function calls it using the necessary arguments.

Program to define a Function for computing Area and calling it

```
def area(length, breath): #define the Function area
```

```
    return length * breath # Body of function Area
```

```
areaofroom = area(10, 5)
```

```
print(areaofroom)
```

```
print(area(10, 10))          #Calling function to compute area
```

➔ Output

```
50
```

```
100
```

There are four major types of functions in Python. These along with the use cases are listed below.

1.10 BUILT-IN FUNCTIONS

Functions that are available in Python, such as print(), len(), sum(), int(), etc.

User-defined functions

Functions created by the programmer to perform specific tasks. These are defined using the def keyword, followed by the function name, parentheses (), and a colon : as illustrated for the function to compute the area().

Lambda functions

A lambda function is a special type of function without the function name. It is a one-line functions defined using the lambda keyword. These are generally used for simple operations. Functions like welcomemessage () discussed above qualify for lambda functions.

Recursive functions

A recursive function in Python are the functions that calls itself within its definition. Each recursive function must have two key components namely a base condition and a recursive step like in any other programming language..

Base Case

This is the condition that stops the recursion and prevents infinite loops. Without this condition, the function can call itself indefinitely, leading to a stack overflow.

Recursive Step

This is where the function calls itself like in calculating Factorial of a number.

Python supports both Method and Function. As we have seen, Methods are dependent on objects and are called by their name. A method definition always includes 'self' as its first parameter and is implicitly passed to an object on which it is invoked. It may or may not return any data. Although, Function is block of code that is also called by its name but function can have different parameters based on the application desired or may not have any at all. If any data parameters) are passed, they are passed explicitly. It may or may not return any data. Finally, Functions don't deal with Class and objects.

Python program to illustrate Recursive Function

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
  
print(factorial(4))
```

→ Output

24

Base Case and Recursive Case of Factorial(n)

Base Case: This is the condition under which the recursion should stop. It is crucial to prevent infinite loops and to ensure that each recursive call reduces the problem in some manner. In the factorial() code illustrated above, the base case is $n == 1$ since when $n == 0$, it returns and comes out of the loop.

Recursive Case: This is the computing part of the function calls to itself. It must eventually lead to the base case. In the factorial example, the recursive case is $\text{return } n * \text{factorial}(n-1)$.

Simple Input and Output in Python

In Python, taking input from the console and displaying the output are fundamental for interacting with users and displaying information while executing any code. The input() function ensures reading data from the console, while the print() function enables displaying output.

Input

The `input()` function prompts the user for a string of text once you execute a program. It takes argument that is displayed as a prompt to the user. The function then waits for the user to enter text and press Enter. The entered text is returned as a string.

Remember that every console input in Python obtained via the `input()` function is always taken as a string. If we have to use the input as a different data type, such as an integer or float, it is necessary to explicitly convert the input using type casting functions like `int()` or `float()`. If the input is intended to be used as a string, no explicit conversion is required. The same is illustrated by a Python program below.

Python program to illustrate use of `input()` function to take user input

```
num = input('Enter a number: ')
print('You Entered:', num)
print('Data type of num:', type(num))
```

➔ Output

Enter a number: 125

You Entered: 125

Data type of num: <class 'str'>

1.11 TAKING MULTIPLE INPUTS FROM CONSOLE

When we require multiple inputs from the user in a single line, splitting the values entered by the user into separate variables for each value is necessary and the same is achieved by using the `split()` method of Python. The same is illustrated by a Python code below.

Python program to illustrate taking multiple inputs at a time

```
x, y = input("Enter two values: ").split()
print("Number of cars: ", x)
print("Number of bikes: ", y)
```

Illustrating Concatenation of strings taken as Console Inputs

```
y, z = input("Enter three values: ").split()
total = y + z
print("Total number of vehicles: ", total)
print("Number of cars is: ", y)
print("Number of bikes is: ", z)
```

➔ Output

Enter two values: 5 6

Number of cars: 5

Number of bikes: 6

Enter three values: 5 6

Total number of vehicles: 56

The two strings 5 and 6 are

concatenated

Number of cars is: 5

Number of bikes is: 6

```
# Typecasting Console Inputs to obtain the Sum
y, z = input("Enter two values: ").split()
total = int(y) + int(z)
print("Total number of vehicles: ", total)
print("Number of cars is: ", y)
print("Number of bikes is: ", z)
```

➔ Output

```
Enter two values: 5 6
Number of cars: 5
Number of bikes: 6
Enter three values: 5 6
Total number of vehicles: 11
Number of cars is: 5
Number of bikes is: 6
```

1.12 EXCEPTION HANDLING

Python exception handling provides us with a backup plan for our code in a situation when something unexpected happens during the execution. Hence it can be said to be an exception that occurs during program execution. For example, dividing number by 0 raises exception called a `ZeroDivisionError`. Whenever these types of runtime errors occur, Python creates an exception object. For serious programmers, exception handling allows developing a robust and reliable code. By planning and handling these types of errors and unexpected situations, we ensure that our code continues to function correctly during unexpected events. With `try`, `except`, and `finally` blocks, Python provides powerful tools for managing errors and maintaining the integrity of our code to create maintainable software applications.

Lets us see an exception situation depicted in a code snippet below. We don't provide for any specific exception handling and hence the output of this prints a traceback to that error along with some details about why the error has occurred as shown below.

```
# Need of Exception Handling
```

```
divide_by_zero = 7 / 0

print(divide_by_zero)
```

➔ Output

ERROR!

Traceback (most recent call last):

File "<main.py>", line 1, in <module>

ZeroDivisionError: division by zero

try...except...finally in Python

In Python, the 'try...except...finally' statement is an extended version of the 'try...except' statement.

Try Block: This is where we write the code that might cause an error or exception condition.

Except Block: This is where we write the corrective backup plan in case an error happens in the try block.

Finally Block

This block always runs irrespective of the fact that an exception occurs or not. It's like putting things right after handling the exception. It guarantees predictable code behaviour that creates maintainable code. Unlike the try-except block, which handles exceptions, the try-finally block focuses on corrective cleanup operations that must occur, ensuring resources are properly released and the intended tasks are completed.

Now let's take up the same code that tries to divide a number by zero and implement the same with try..except..finally.

Program with no exception and role of Finally

try:

```
number = 7 / 1
```

except ZeroDivisionError:

```
print("Can't divide a number by zero")
```

finally:

```
print('This part of code is always executed')
```

➔ Output

This part of code is always executed

Program with exception and the role of Finally

try:

```
number = 7 / 0
```

except ZeroDivisionError:

```
print("Can't divide a number by zero")
```

finally:

```
print('This part of code is always executed')
```

➔ Output

Can't divide a number by zero

This part of code is always executed

1.13 Built-in Exceptions in Python

For the sake of insight, the table below illustrates the built in exceptions that are available in Python (in alphabetic order).

ArithmeticError	error due to numeric calculations
AssertionError	when an assert statement fails
AttributeError	attribute reference or assignment fails
Exception	Base class for all the exceptions
EOFError	when input() method encounters an "end of file" condition (EOF)
FloatingPointError	Due to a floating point calculation failure
GeneratorExit	when a generator is closed
ImportError	when an imported module is not found
IndentationError	indentation incorrect
IndexError	index of a sequence does not exist
KeyError	key not found in a dictionary
KeyboardInterrupt	Due to pressing Ctrl+c, Ctrl+z or Delete
LookupError	when errors raised is not found
MemoryError	Program memory runs out
NameError	variable does not exist
NotImplementedError	abstract method requires an inherited class to override any method
OSError	system related operation causes an error
OverflowError	result of a numeric calculation is too large
ReferenceError	reference to weak object does not exist
RuntimeError	error that do not belong to any specific exceptions
StopIteration	next() method of an iterator has no values left
SyntaxError	syntax error occurs
TabError	Due to indentation consists of tabs or spaces
SystemError	Due to a system error
SystemExit	Calling the sys.exit() function
TypeError	Combining two different types
UnboundLocalError	Referencing local variable before assignment

UnicodeError	unicode problem found
UnicodeEncodeError	unicode encoding problem found
UnicodeDecodeError	unicode decoding problem found
UnicodeTranslateError	unicode translation problem found
ValueError	wrong value found in a specified data type
ZeroDivisionError	second operator in a division is found to be zero

1.14 ITERATORS AND GENERATORS

Iterators and generators in Python provides the programmers to iterate over the data one item one at a time without having to load all of the data together into memory at once. This is particularly helpful when working with large datasets for efficient memory management.

Iterators in Python

An iterator is an object that allows the programmers to traverse / iterate or loop through a collection of items like a list one item at a time.

The Iterator Protocol

Iterators follow certain rules that constitutes the iterator protocol. This consists of two main methods listed below:

`__iter__():`

This method initializes the iterator and returns it.

`__next__():`

This method returns the next item in the collection. If there are no more items, it issues a `StopIteration` error to signal that the iteration is complete.

class IteratorExample:

```

    def __init__(self, item):
        self.item = item                # To iterate over the given list
        self.index = 0                  # Initializing the starting index in the
list
    def __iter__(self):
        return self                    # Returns the iterator object itself
    def __next__(self):
        if self.index < len(self.item): # To check if there are more items in
the list
            # Get the current item

```

```

        value = self.item[self.index]

        self.index += 1

        return value                                # Return the current item

    else:

        raise StopIteration

# Usage of IteratorExample

prime_list = [1, 2, 3, 5, 7, 11] # A simple list

iterator = IteratorExample(prime_list)           # Create an iterator for the list

# Loop through the items using the Iterator

for number in iterator:

    print(number) # Print each item

```

→ Output

```

1
2
3
5
7
11

```

1.15 GENERATORS IN PYTHON

A generator is a simpler way to create an iterator using a function that outputs certain values. When a generator function is called, it returns a generator object, which behaves like an iterator. Each time the `next()` function is invoked on the generator object, the function body executes until a `yield` statement is encountered. The value produced by `yield` is returned, and the function's state is saved. Subsequent calls to `next()` resumes the execution of the function from the last saved state, until the function exits or raises a `StopIteration` exception.

```

def generatorExample(item):
    for number in item:                                # Loop through each item in
the data                                              the data
        yield number                                # Yield (return) the current
item and pause                                     item and pause

# Usage of my_generator
prime_list = [1, 2, 3, 5, 7, 11]                    # A simple list
generator = my_generator(prime_list)                # Create a generator

for number in generator:                            # Loop through the items produced by the
generator                                           generator
    print(number)                                # Print each item

```

➔ Output

```
1
2
3
5
7
11
```

So, the above code shows how `yield()` pauses the function execution and returns a value. It saves the variable values and the function execution point and resumes from this point when the generator is called.

1.16 ADDITIONAL PYTHON CONVENIENCES

Python's convenience lies in its vast number of libraries for Data Science (statistical), Machine learning, Deep learning and NLP models illustrated in tables below. Python's widespread use is not only because of its simplicity, it is also because of its various modules and libraries, each tailored to make a developer's coding efficient and scalable code.

Python Libraries for Data Science

Library Name	Use
NumPy	scientific computation
Pandas	data analysis, data manipulation, and data cleaning
Matplotlib	interactive, and animated Python visualizations
Seaborn	data visualization framework

Python Machine Learning Libraries

Library Name	Use
LightGBM	open-source gradient boosting library that employs tree-based algorithms
XGBoost	implementation of machine learning algorithms within the gradient boosting framework
Statsmodels	estimate various statistical models, conduct statistical tests, and do statistical data exploration
Optuna	automate hyperparameter searches

Python Deep Learning Libraries

Library Name	Use
TensorFlow	end-to-end open-source platform for machine learning
PyTorch	optimized tensor library for deep learning using GPUs and CPUs, and is considered to be an alternative to TensorFlow
Keras	Most popular and supports nearly all models of a neural network, including convolutional, embedding, pooling, recurrent networks

Python Libraries for Natural Language Processing

Library Name	Use
NLTK	parse human language data - called “a wonderful tool for teaching, and working in, computational linguistics using Python”
spaCy	natural language processing library in Python - excels at large-scale information extraction tasks
Gensim	topic modeling, document indexing, and similarity retrieval with large data set
Hugging Face Transformers	Pre-trained models for text, audio, images and multimodal

1.17 SCOPES AND NAMESPACES

A namespace is a collection of names where we have a mapping of every name we have defined to corresponding objects in the code. An object can be a variable or a method. Name represents a unique identifier and Space refers to the scope of the name. Python itself maintains a namespace in the form of a Python dictionary. At the highest level of hierarchy, we have built-in name space like print(). This is followed by Global name space. When a user creates a module, a global namespace is created and further, when we write local functions, it creates the local namespace. Thus, a local namespace is created when a function is called and this has all the names defined in it. Hence, the built-in namespace encompasses the global namespace and the global namespace encompasses the local namespace. The same is illustrated by a Python program below.

```
# Program to illustrate usage of Global, Local and Nested Namespaces

global_var_val = 2                # global_var represents global namespace
def outermost_function():
    outer_var_val = 4              # This variable is in the local namespace
    def inner_function():
        inner_var_val = 6         # This is in the nested local namespace
        print(inner_var_val)
    print(outer_var_val)
    inner_function()
print(global_var_val)             # Prints the value of the global variable
outermost_function()             # Calls the outer function that includes local and nested local
variables and prints it
➔ Output
2
4
6
```

1.17.1 Scope of Objects in Python

One cannot access any particular object from anywhere in the code, the same is defined by the scope of the object. Scope refers to the part of the program from where a particular Python object can be referenced. This translates to the portion of a program from where a namespace can be accessed directly without any prefix. At any point of interpreting a code, there are at least three nested scopes mentioned below.

Scope of the function in reference which contains local names searched in local namespace

Scope of the module which has global names searched in the global namespace

Outermost scope which has built-in names and namespaces

It is worth mentioning that the lifetime of a namespace depends upon the scope of objects being referenced. If the scope of an object ends, the lifetime of that namespace also comes to an end.

1.17.2 Modules and the Import Statement

Most of us have used “#include” while writing the C or C++ codes. This file includes numerous methods / functions that makes coding easier. Similarly, a Module is a file that contains methods / functions code to perform various tasks. These modules could be built in facility of python or one can also create own Module that stores all the methods / functions that you frequently use during coding. A module may contain variables, functions, classes etc. As the LoC (Lines of Code) increases, program size grows bigger. Thus, instead of putting everything in a single program file, we use modules to call the required methods / functions as per the functionality required. This makes our code understandable and maintainable.

User Defined Modules

Let us create a module that stores function for multiplying two numbers and save the file as multiplyexample.py. Now, one can import the definitions that exist inside a module to another module or the interactive interpreter in Python where you execute the code. This is done by the use of ‘import’ keyword.

Python program to create a sample Module

```
def multiply(a, b):  
    result = a * b  
    return result
```

Now, to use this module, we can go to our Python prompt and type:

```
Import multiplyexample  
multiplyexample.multiply(5,4) # Calling multiply function in multiplyexample
```

➔ Output

20

Similarly, the Python library built-in modules or any specific function in these modules can be used by the “import” statement.

Illustrating usage of importing Math Module of Python

```
import math  
cube = pow(2, 3)           # Finding Cube of a number  
print(cube)
```

➔ Output

8

To make efficient usage of the resources, one can import the specific functions or variables that is required in our code using the from keyword instead of importing the entire module. The same is illustrated below.

Illustrating usage of importing specific function from Math Module of Python

```
from math import pow
cube = pow(2, 3)          # Finding Cube of a number
print(cube)
```

➔ Output
8.0

One can shorten the name of the module being imported although it shall promote cryptic programming. Nevertheless, this is achieved by the use of “alias” as illustrated here.

```
import math as aa
cube = aa.pow(2, 3)
```

1.18 SUMMARY

In recent years, Python has become a language of choice since it has simple syntax, easier code readability, user friendly and can be used with wide variety of applications. It has extensive library, large active user community, cross-platform compatibility and beginner-friendly design philosophy. To understand its acceptability, for web and software development, Python is behind many websites and applications. For understand the hidden data patterns, Data analysts use Python’s libraries to process and visualize huge datasets. Python is a dynamic, interpreted (can also be referred to as byte code-compiled) language. This means that prototyping can be very quick while testing the code.

Python is an object-oriented language and classes form the basis for all its data types. Classes are a means of bringing together data and functionality together. In Python, an identifier is a name given to a programming element, such as a variable, function, class, or module. It allows programmers to refer to these elements within their code. It is worth mentioning that Python does not have a built-in do-while loop like other programming languages. However, it is possible to exhibit the similar effect of a do-while loop using a while loop along with a break statement. The keyword “def” is used to create a function. A function is a block of reusable code that performs a specific task.

Every console input in Python obtained via the input() function is always taken as a string. If we have to use the input as a different data type, such as an integer or float, it is necessary to explicitly convert the input using type casting functions like int() or float(). If the input is intended to be used as a string, no explicit conversion is required. Python exception handling provides us with a backup plan for our code in a situation when something unexpected happens during the execution. Iterators and generators in Python provides the programmers to iterate over the data one item one at a time without having to load all of the data together into memory at once. This is particularly helpful when working with large datasets for efficient memory management. Python's convenience lies in its vast number of libraries for Data Science (statistical), Machine learning, Deep learning and NLP models.

The built-in namespace encompasses the global namespace and the global namespace encompasses the local namespace. The modules could be built in facility of python or one can also create own Module that stores all the methods / functions that you frequently use during coding. A module may contain variables, functions, classes etc. As the LoC (Lines of Code) increases, program size grows bigger. Thus, instead of putting everything in a single program file, we use modules to call the required methods / functions as per the functionality required. This makes our code understandable and maintainable.

Check your Progress

1. What are the benefits of using Python?
2. What operators does python support?
3. What are the common built-in data types in Python?
4. Where and how is a Python function defined?
5. What is a Try block in Python?
6. What is recursion in Python?
7. What is meant by module in python?
8. Write few names of Python built-in modules?
9. What are the common built-in data types in Python?
10. What is Scope in Python?
11. How good is Python for data analysis?
12. What are Python namespaces? Why are they used?
13. Is Indentation Required in Python?
14. What is a namespace in Python?
15. What is the difference between .py and .pyc files?

1.19 Further Readings

1. Python Essentials You Always Wanted to Know: Beginner's Guide to Python Programming, Data Structures, Data Analytics with Hands-On Coding Exercises

by Shawn Peters and Vibrant Publishers

2. Python Programming : Beginners Guide Paperback

by Sundarajan M (Author), Mani Deepak Choudhry (Author), Jeevanandham S (Author), Akshya Jothi (Author)

3. Python Crash Course: A Hands-On based Introduction to Programming by Eric Matthes; No Starch Press (Publisher); ISBN-10: 1718502702; ISBN-13: 978-1718502703
4. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter by Wes McKinney; O'Reilly (Publisher); ISBN-10: 9355421907; ISBN-13: 978-9355421906

Reference Websites

<https://pythoninstitute.org/python-essentials-1>

<https://pythoninstitute.org/python-essentials-2>

